

AutoBench

Architecture & Design Overview

A pure-Python, HTTP-only service that deploys and evaluates agent benchmarks across multiple cluster-specific Rossoctl instances.

Last modified 2026-09-17 01:07 EDT

Agenda

What this deck covers, in order

1

Overview & key design decisions

what the Service is, and the seven choices that shape it

2

Auto-benchmarking design motif

the shape of the system, before the wiring detail

3

Architecture

client, Service, and the per-cluster instance selected by JWT iss

4

Architecture with workload specific components

4.1 inside the workload: sidecar, user simulator, IBAC judge · 4.2 how interception is wired

5

The two-token auth model

why the caller's token is never forwarded upstream

6

Benchmark catalog & run lifecycle

6.1 the three benchmarks and the REST flow that drives them · 6.2 what each image bakes in

7

The three benchmarks — what they measure

7.1 the ladder · 7.2 what each stresses · 7.3 the traps · 7.4 picking one · 7.5 what it costs

8

The canonical 12-run matrix

8.1 what the 12 runs parameterize · 8.2 what they measured

9

Cross-platform & plugin overhead

9.1 OpenShift vs KinD, like-for-like · 9.2 what AuthBridge costs

10

What the Service can & cannot enact

the HTTP-only boundary, made explicit

1. Overview & Key Design Decisions

What it is

Deploys a benchmark's MCP tool + A2A agent, runs the evaluation, reads results, and exports them — all over HTTP.

Objectives

- Automate benchmarking lifecycle operations
- Cross-user/cluster sharing of benchmark run results
- Easy to configure in-cluster & cross-cluster benchmark runs
- Asynchronous parallel benchmark runs
- Secure, scalable, auditable & resilient

3 benchmarks

- gsm8k — single-turn: one prompt in, one answer out, no dialogue partner
- tau2 — multi-turn: a server-side user simulator LLM plays the customer
- appworld — long-horizon: many API calls across simulated apps

Client contract

- Point at a hostname, send Authorization: Bearer <caller JWT> — same from kind to prod

Key design decisions

Pure-Python, HTTPS-only to Rossoctl

No shell-out: smaller image, no injection surface, structured errors, testable.

Two-token auth model

Caller JWT attributes + routes only (never forwarded); Service mints its own benchmarker (ROPC) token to Rossoctl.

iss is the trust anchor

The JWT issuer selects the per-instance config — no separate instance argument, so no confused-deputy escape.

Per-request ROPC login

Fresh Service token every request — no expiry handling.

iss-keyed per-instance config

One file per issuer: Rossoctl URL, benchmarker cred, MLflow/S3, optional Keycloak backchannel + workload route templates.

Enact only what the API allows

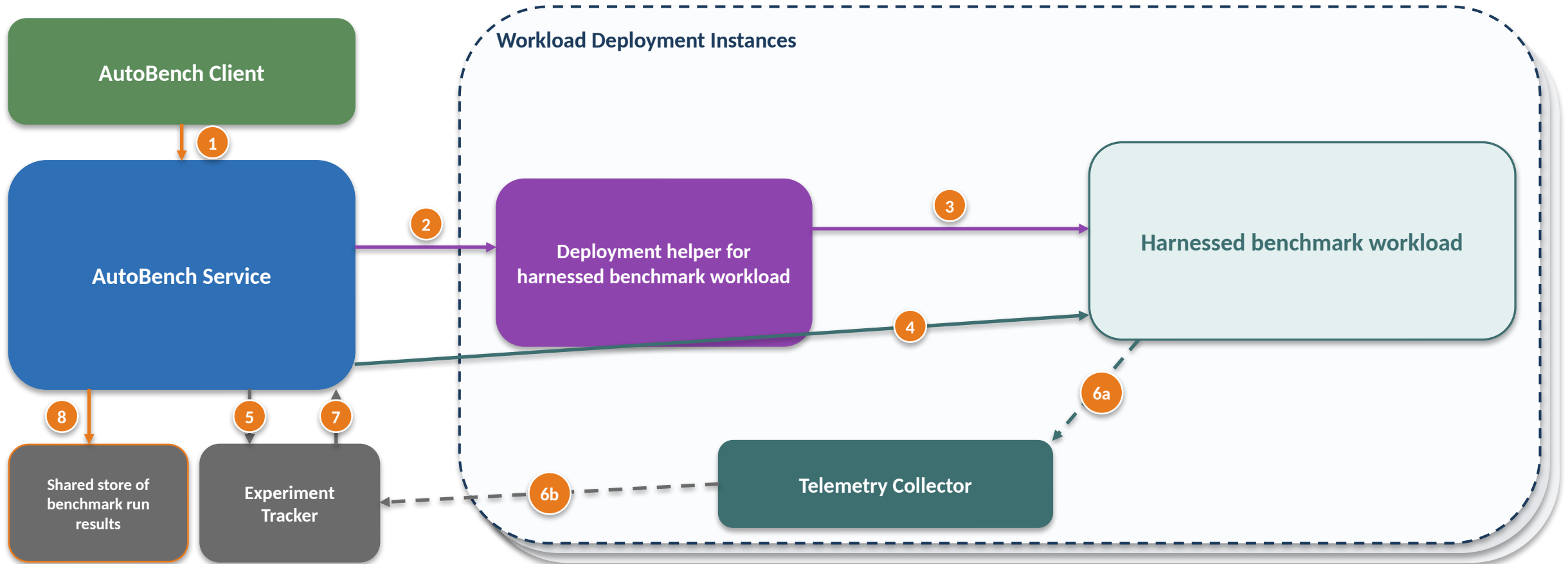
Deploy/run/report/export — now including AuthBridge plugin presets (layer-3). Only workload Secrets remain out-of-band: prechecked and reported (424), never silently ignored.

MLflow with Service; S3 in the cloud

MLflow is co-located with the Service (per-service traces it emits + reads), not in the workload cluster; S3 is an external cloud service — the shared cross-service sink.

2. Auto-Benchmarking Design Motif

One client, one Service, N workload deployments — and every result ends up in one place

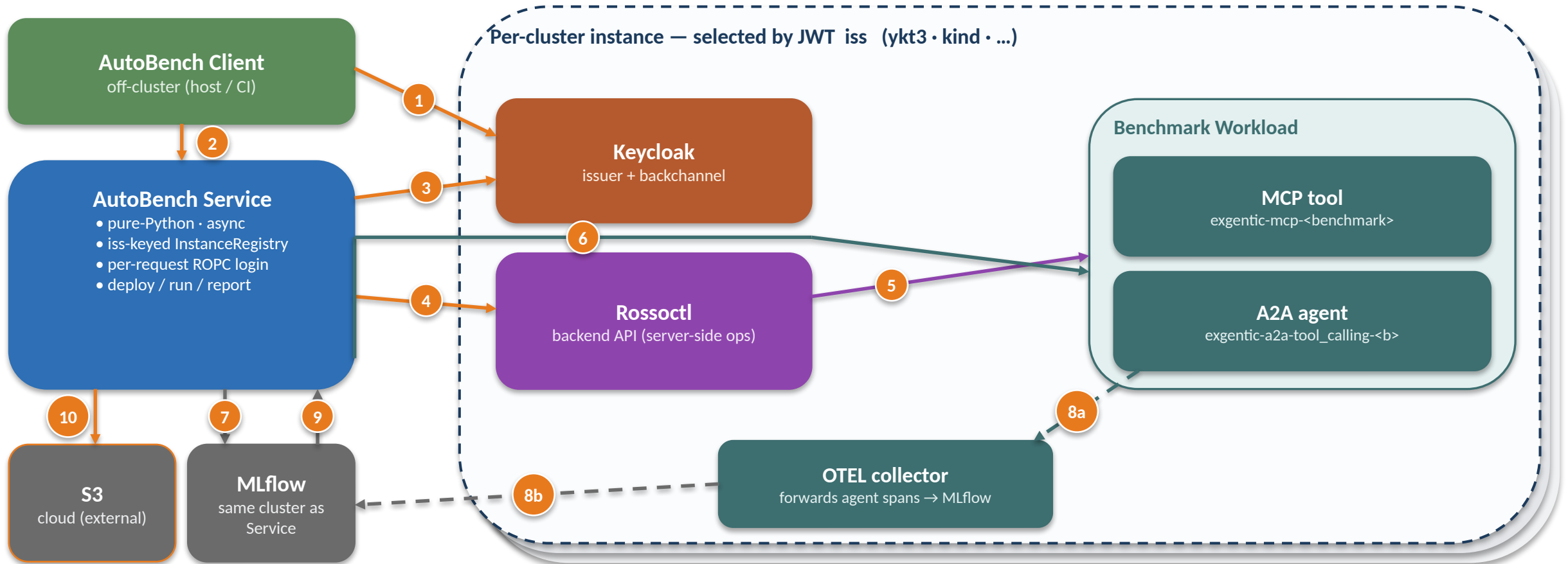


- 1 Client → Service
- 2 Service → deployment helper: create the harnessed workload
- 3 helper → harnessed workload
- 4 Service → harnessed workload: run and monitor benchmark execution

- 5 Service → Experiment Tracker: emit the trace
- 6a workload → Telemetry Collector · 6b collector → Tracker
- 7 Experiment Tracker → Service: read the records back
- 8 Service → shared store: export the run's artifacts

3. Implementation Architecture: Kubernetes-Based Benchmark Workloads

Relations between client, service, cluster-specific Keycloak / Rossoctl / workload, MLflow & S3

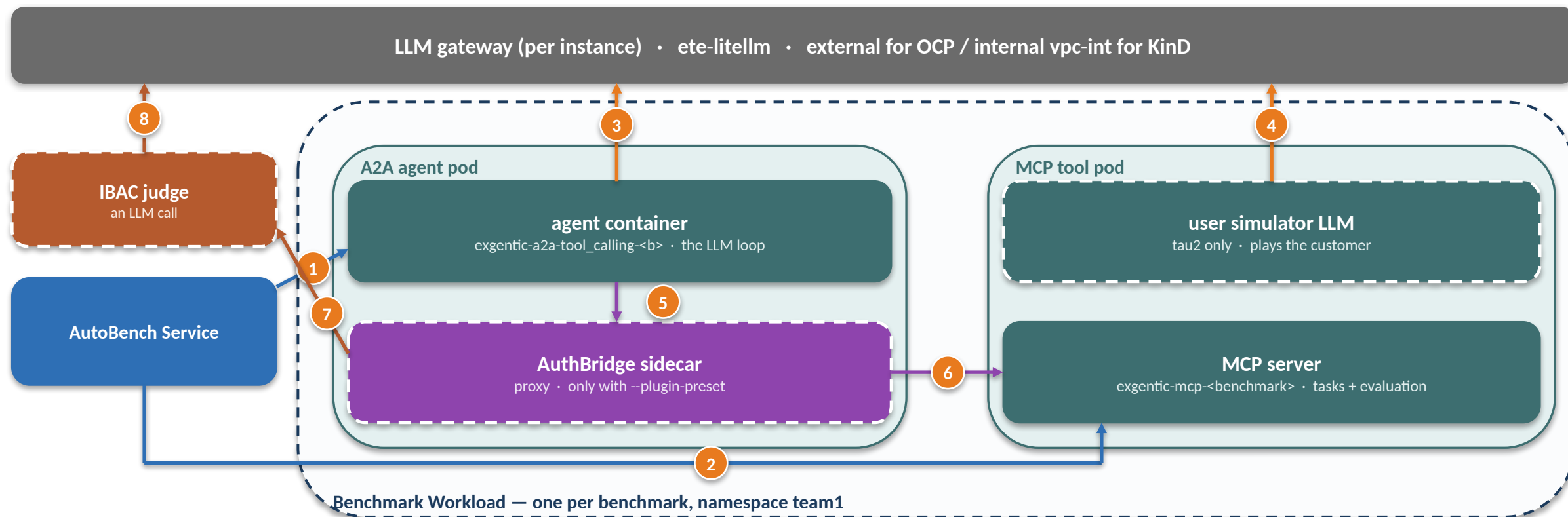


- 1 Client logs in to Keycloak (ROPC) → caller JWT
- 2 Client → Service: Authorization: Bearer <caller JWT>
- 3 Service validates JWT (JWKS) + mints its own benchmark token (ROPC, via backchannel)
- 4 Service → Rossoctl: deploy / list agents + tools
- 5 Rossoctl creates the MCP tool + A2A agent in-cluster

- 6 Service drives the MCP session + the A2A call itself; the agent only issues execute_tool inside that session
- 7 Service emits the Agent.Session trace → MLflow (first)
- 8a A2A agent → OTEL collector · 8b collector → MLflow (optional workload spans — off by default)
- 9 Service reads back Agent.Session traces (MLflow)
- 10 Service exports run.json / report.* (S3)

4.1 Architecture with Workload Specific Components

Inside the Benchmark Workload — what every benchmark has, and what only some of them add



Which components a benchmark actually gets

benchmark	MCP tool image	LLMs per task	tool calls
gsm8k	exgentic-mcp-gsm8k	1 (agent)	~1
tau2	exgentic-mcp-tau2	2 (+ user simulator)	~11
appworld	exgentic-mcp-appworld	1 (agent)	~15
+ any preset	unchanged	+1 IBAC judge call per action	—

- 1 Service → agent: `send_prompt`, once per task
 - 2 Service → MCP server: `list_tasks`, `create_session`, `evaluate_session`, `delete_session` (the verdict)
 - 3 agent → gateway: N real chat calls per task
 - 4 tau2 only: the user simulator is a 2nd LLM
 - 5 with a preset: the agent's MCP calls go via the sidecar
 - 6 sidecar → MCP server, once authorized
 - 7 sidecar → judge, per `isAction` call
 - 8 the judge is itself an LLM call
- Dashed outline = present only in some configurations.
No preset: 5 + 6 become one direct agent → MCP call.

4.2 How Interception Is Wired — and How It Silently Isn't

The sidecar sees traffic only because of two env vars; a plausible config takes it back out of the path

Interception is a blanket forward proxy plus an allowlist — not a tool-aware hook. Three settings decide what the sidecar actually sees:

HTTP_PROXY = HTTPS_PROXY = http://127.0.0.1:8081

the OPERATOR injects this into the agent pod when AuthBridge is enabled — 127.0.0.1:8081 IS the sidecar, so by default every outbound HTTP call the agent makes is captured

no_proxy = LLM gateway, OTEL collector, Keycloak

instance config — the hosts that stay DIRECT. This is the only reason flow 3, the agent's own chat calls, does not go through the sidecar

judge_inference: false

IBAC's own default — even proxied inference is not judged. BOTH this and no_proxy would have to change for the agent's model calls to be authorized

✓ Through the sidecar

- agent → MCP tool calls (flows 5 → 6) — the interception point
- tools/call → isAction=true → one IBAC judge call each (flow 7)
- inbound A2A message/stream → a2a-parser, isAction=false, so the Service's send_prompt is logged but never judged

✗ Direct — the plugins never see it

- agent → LLM gateway (flow 3) — named in no_proxy, and judge_inference: false would exempt it even if proxied
- Service → MCP server (flow 2) — a different pod; the Service does not dial through the agent's proxy
- MCP tool → gateway (flow 4, tau2's user simulator) — different pod

A ready sidecar proves nothing about enforcement — the double bypass we actually shipped:

1. workload_llm.disable_proxy: true makes the SERVICE inject HTTP_PROXY="" first

2. the operator adds a var only when ABSENT → it skips HTTP_PROXY, but still sets HTTPS_PROXY

3. MCP_URL is http:// → tool calls take the empty HTTP_PROXY and go direct

4. no_proxy carried .svc.cluster.local → the same calls are excluded a second time

Result: sidecar ready = True, pipeline ConfigMap rendered, plugins loaded — and the judge called ZERO times, on both clusters.

Verify by COUNTING judge chat completions across a run. IBAC emits no log line of its own, so a healthy sidecar log proves nothing — and initialize / tools/list are isAction=false and legitimately never reach the judge, so a zero count only means something if real tool calls occurred. Working config: leave disable_proxy off, keep the NAMED hosts in no_proxy, drop the WILDCARDS (.svc, .svc.cluster.local).

5. The Two-Token Auth Model

Why the caller's token is never forwarded upstream

Caller JWT (inbound)

Presented by the client as Authorization: Bearer.

- Used ONLY to attribute (preferred_username) and route (iss → instance)
- Signature validated via the issuer's JWKS
- aud / exp deliberately lenient in the dev/ops context
- **NEVER forwarded to Rossoctl**

benchmarker is special:

- A caller JWT with preferred_username == benchmarker authorizes config ops (GET/PUT /config)

Service token (outbound)

Minted by the Service per request via ROPC (password grant) using the instance's benchmarker credential.

- Always fresh → no expiry handling
- Presented to Rossoctl for all cluster-facing ops
- **The Service acts under its OWN identity, not the caller's**

Backchannel split (iss ≠ dialed host):

- When the iss host is unreachable (in-cluster kind), JWKS + token come from a configured backchannel URL; iss is matched, never dialed

Implication: at the Rossoctl / cluster layer every action appears as the Service's identity — so per-user attribution & audit live in the Service, keyed on (iss, preferred_username).

6.1 Benchmark Catalog & Run Lifecycle

Catalog (static registry)

gsm8k

single-turn · needs hf-secret + openai-secret

tau2

multi-turn · user-simulator LLM runs server-side in the MCP pod · raise timeout

appworld

long-horizon across simulated apps · runs e2e · raise task_timeout_seconds

Lifecycle (REST)

POST /benchmarks/{n}/deploy

create MCP tool + A2A agent → 201

GET /benchmarks/{n}/status

poll until tool_ready & agent_ready

POST /benchmarks/{n}/runs

precheck → 202 run_id (409 not deployed · 424 secret missing)

GET .../runs/{id}

poll pending → running → succeeded / failed

GET .../runs/{id}/report

MLflow records + artifacts (409 if MLflow unset)

download from S3

run.json · report.* · token_report.* · span_report.* · manifest.json

6.2 What Each Benchmark Bakes In — and What It Needs From Us

The MCP image carries the benchmark itself; only the credentials and quirk overrides come from tool_env

	MCP tool image	what the image bakes in	what tool_env must add
gsm8k	exgentic-mcp-gsm8k	the HuggingFace dataset loader — 8.5K problems, fetched at pod startup	HF_TOKEN (from hf-secret), plus EXGENTIC_SET_BENCHMARK_RUNNER=direct
tau2	exgentic-mcp-tau2	the tau2-bench library + its retail domain (114 tasks), and a user-simulator LLM	OPENAI_API_KEY (from openai-secret) + EXGENTIC_SET_BENCHMARK_ACTION_TIMEOUT=1000 — the simulator makes its own inference calls (flow 4)
appworld	exgentic-mcp-appworld	the whole app-suite sandbox (exgentic install --benchmark appworld)	nothing beyond BENCHMARK_NAME — upstream's .env.appworld is explicitly empty

One agent image serves all three

exgentic-a2a-tool_calling:latest is the only key in every agents={...} map; per benchmark it only gains a -<benchmark> name suffix. So the benchmark lives in the MCP pod, and the subject under test is the same binary every time.

The LLM base is never baked in

OPENAI_API_BASE is injected per deploy from the instance's workload_llm.api_base, and tau2's simulator model from the run's model — a deploy with no gateway configured is rejected with 422 rather than defaulting.

Two traps in that last column: appworld REJECTS the same action-timeout override tau2 needs and crashes at startup with "Unknown benchmark override 'action_timeout'" — the env is per-benchmark, not a shared default. And hf-secret must EXIST for gsm8k even though the dataset is public: without it the MCP pod sits in CreateContainerConfigError and the agent crash-loops. Missing secrets surface as a 424 on the run precheck, naming exactly what to provision.

7.1 The Three Benchmarks — a Difficulty Ladder

Not interchangeable suites: each costs ~an order of magnitude more than the last

	gsm8k	tau2	appworld
What it tests	multi-step arithmetic	multi-turn dialogue + tools	long-horizon app automation
Task rows measured	172 of 172	60 of 60	35 of 50 (15 timed out)
Rows with lost telemetry	0	0	0
Pass rate	0.97	0.83	0.00
Input tokens / task	341	90,902	269,953
Output tokens / task	180	2,061	25,140
LLM calls / task	1.1	11.4	29.2
Tool calls / task	1.1	11.4	14.6
Median task latency	4.9 s	84 s	264 s
Slowest task seen	39 s	136 s	592 s
Model we use	gpt-5-mini	claude-sonnet-5	gemini-2.5-pro
Task pool	8.5K (HuggingFace)	114 (retail domain)	grouped scenarios

The scale gap is the headline: a tau2 task costs ~267x the input tokens of a gsm8k task, an appworld task ~792x. A 50-task gsm8k run is a minute; a 20-task appworld run is 30-45 minutes and millions of tokens. Budget by benchmark, not by task count.

7.2 What Each Benchmark Stresses

Why all three are in the matrix, and what a result from each does and does not tell you

gsm8k — the canary

- One prompt in, one answer out — no dialogue partner.
- ~1 real LLM call + 1 tool call: the simplest agentic loop.
- Stresses almost nothing about the platform — which is the point.
- A failure here means INFRASTRUCTURE: deploy, auth, LLM reach, telemetry.
- Saturates near 1.0, so it cannot discriminate models. Never read it as one.
- Deterministic enough that task 0 costs 320 input tokens on every cluster — we use that to prove two environments are comparable.
- Cheap enough to run often: a 50-task leg is 25 K tokens — 0.4% of the whole 12-run matrix's bill, and less than a SINGLE appworld task (see 7.4).

tau2 — the discriminator

- Multi-turn: a server-side USER SIMULATOR LLM plays the customer.
- Two models talk to each other, plus ~11 tool calls per task.
- Domain is retail (114 tasks) — the library default, never recorded in artifacts.
- The only leg where model choice dominates: 0.1 with gpt-5-mini vs 0.9 with claude-sonnet-5 on the SAME 10 tasks.
- Episodes are nondeterministic: the same 10 tasks scored 0.90 on one cluster and 1.00 on the other, in the same matrix.
- At n=10 one task moves pass_rate by 0.10 — it cannot resolve less than that.
- 27% of a task's wall time sits INSIDE tool calls — that is the simulator generating, server-side, and its tokens appear in no report of ours.

appworld — the stress test

- Realistic chores across simulated apps: discover APIs, chain many calls.
- ~29 LLM calls, ~15 tool calls, ~270k input, ~4.5 min per task.
- Evaluation is PROGRAMMATIC unit tests over final app state — no LLM judge.
- Pass rate 0.0 is honest, not broken: runs complete and tokens record — the agent just does not finish the job.
- 34 of 35 measured tasks call finish: it believes it is done and the assertions disagree. No verification pass.
- Its job here is pipeline stress: long contexts, big traces, real timeouts.
- Two model calls per tool call — tool shortlisting, which only bites with an API surface this large. The first carries 68% of the input tokens.

7.3 Reading the Numbers — Five Things That Mislead

Every one of these cost us a wrong conclusion first

1. **pass_rate = evaluated_pass / total**

A task that ERRORS before evaluation counts as not passed, so a low rate can mean “failed the task” or “never got judged”. Check the error column too.

2. **The llm column changed meaning between agent versions — never compare across it**

Agents up to exgentic 0.3.5.dev131 issued a max_tokens=1 capability probe counted as a chat span, so THEIR llm=2 on gsm8k means ONE real call. From dev145 the probe is an unbilled GET /v1/models that emits no span — verified absent across all 1,895 chat spans of these two matrices. Current runs count real calls 1:1, with no offset to subtract.

3. **Implausibly small tokens = lost telemetry, and tokens == 0 will not catch it**

Use the structural test: llm <= 1 with tool >= 2 is impossible, since each tool call needs a preceding model turn. A zero-check misses the cases that matter — while the probe existed it was the span that SURVIVED the loss, carrying 8/1 on claude-sonnet-5 and 1/0 on gemini. The structural pair holds across agent versions; it is what the generators use.

4. **Output varies MORE than input, in most runs**

Measured OUT CV > IN CV in 15 of the 22 legs that ran more than one task. gsm8k's prompt is near-constant while answer length swings; only long-horizon appworld inverts it, in all four of its legs. Do not infer a direction from the mechanism — read the CV.

5. **Task selection is deterministic**

A run takes the first max_tasks tasks, so the same task_id is the same task across runs and clusters, and a smaller run is a prefix of a larger one. That is what makes cross-platform comparison like-for-like — six legs matched to the byte.

7.4 Picking a Benchmark — and What That Leg Costs

The actionable summary: what each benchmark is FOR, and the token bill it hands you

If you want to ...	use
check a cluster / deploy / auth / telemetry path works	gsm8k, 1–10 tasks
exercise concurrency and volume cheaply	gsm8k, 50 tasks at p=4
compare models meaningfully	tau2 — it discriminates; gsm8k saturates at ~1.0
stress long contexts, long tasks, timeouts	appworld
get a fast signal that nothing regressed	gsm8k — if it fails, stop and fix infrastructure

leg (v1.28)	tokens OCP	tokens KinD
#1 gsm8k, 1 task	470	790
#2 gsm8k, 10 tasks	5.1 K	5.1 K
#3 gsm8k, 50 tasks p=4	25 K	24 K
#9 tau2, 10 tasks	1.01 M	1.04 M
#10 tau2, 20 tasks p=4	1.74 M	1.78 M
#11 appworld, 5 tasks	1.49 M	0.93 M
#12 appworld, 20 tasks p=4	5.71 M	2.20 M
all 12 legs	10.0 M	6.0 M

Budget by benchmark, not by task count

The eight gsm8k legs together are 0.4% of the matrix's token bill (0.8% on KinD). appworld's two legs are 72% of it (52% on KinD). A 50-task gsm8k leg is cheaper than a SINGLE appworld task — 25 K tokens against 295 K.

Two legs of the same size are not the same bill

#12 cost 2.6x more on OpenShift than on KinD for the same 20 requested tasks: appworld turn counts are nondeterministic, and the slower cluster's tasks ran longer before the 600 s timeout. Size appworld on YOUR cluster.

And the totals UNDERSTATE it three ways: a task killed by the per-task timeout burns tokens but leaves no report row, so appworld's 15 timed-out tasks are missing from the numbers above. tau2's user simulator runs in the MCP pod, which is not instrumented — its inference is billed by the gateway and counted nowhere here. And a leg that replays the gateway's completion cache re-reports stored usage for calls that were never made upstream.

7.5 The Cost Model — Tokens, Models, Money

What a task costs, what a model choice costs, and the one ratio that decides it

per task, pooled	gsm8k	tau2	appworld
LLM calls	1.10	11.38	29.20
input tokens	341	90,902	269,953
output tokens	180	2,061	25,140
total tokens	520	92,963	295,093
x a gsm8k task	1x	179x	567x
input share of tokens	66%	98%	92%
median task latency	4.9 s	84 s	264 s
... of it inside model calls	90%	58%	96%
pass rate	0.97	0.83	0.00
tokens per PASSED task	537	112 K	no finite value

same 5 gsm8k tasks, p=4	gpt-4.1	gpt-5-mini
pass rate (OCP / KinD)	0.80 / 1.00	1.00 / 1.00
LLM calls per task	2.8 – 3.0	1.0
input tokens per task	775 – 837	313
output tokens per task	57 – 63	137 – 355
total tokens per task	832 – 900	450 – 668
median task latency	10.4 – 10.8 s	11.2 – 16.4 s

The token ranking and the money ranking disagree

The reasoning model answers in ONE call; gpt-4.1 needs ~3 tool round-trips, so it sends 2.6x the input but emits a quarter of the output. Equating the two bills solves for break-even at output:input $\approx 2.7x$ (the platforms bracket it, 1.8x–5.8x). Priced above that ratio gpt-4.1 is cheaper; below it, gpt-5-mini.

cost per task = (input tokens x P_{in} + output tokens x P_{out}) / 1 M — we publish the token counts, you supply your own rates; our gateway does not bill us, so no dollar figure here would be ours to quote. Two consequences of the input-share row: for tau2 and appworld the bill IS the input side, so the cost driver is turn count and context compounding, not verbosity — and a cheaper-input model beats a terser one. For gsm8k, output is a third of the tokens and reasoning effort moves it 2.6x between clusters.

8.1 The Canonical 12-Run Matrix

One fixed set of 12 request bodies — the same on every platform, every version

#	benchmark	tasks	parallel	what it varies
1	gsm8k	1	1	baseline — the smoke test
2	gsm8k	10	1	volume, still serial
3	gsm8k	50	4	volume + concurrency
4	gsm8k	5	4	model swap → Azure/gpt-4.1
5	gsm8k	5	4	AuthBridge preset: auth-only
6	gsm8k	5	4	AuthBridge preset: ibac-only
7	gsm8k	5	4	AuthBridge preset: full (enforce)
8	gsm8k	5	4	full + per-plugin override ibac:observe
9	tau2	10	1	multi-turn + user simulator
10	tau2	20	4	multi-turn under concurrency
11	appworld	5	1	long-horizon, gemini-2.5-pro
12	appworld	20	4	long-horizon under concurrency

Why a FIXED matrix: task selection is deterministic, so the same leg run anywhere executes the same tasks in the same order. That is what makes a difference attributable to the platform rather than to the workload. Every leg deploys fresh — originally because a warm agent lost telemetry (fixed in agent dev145), now because only a NEWLY CREATED pod re-pulls :latest. Driven by one command (reference/run-12.py): ~1h35m of leg wall time per platform, plus the deploys and ~33 min of gateway-cache gaps. The generated documents in docs/results/ derive every number from the mirrored artifacts.

8.2 What the 12 Runs Measured

v1.28 on OpenShift (Service on ykt3, workloads on ykt2) — 141 tasks, all 12 legs succeeded

#	bench	pass	err	wall	input tokens
1	gsm8k	1.00	0/1	7 s	320
2	gsm8k	1.00	0/10	41 s	3,166
3	gsm8k	1.00	0/50	46 s	15,684
4	gsm8k	0.80	0/5	16 s	3,874
5	gsm8k	1.00	0/5	25 s	1,564
6	gsm8k	0.60	2/5	39 s	1,258
7	gsm8k	0.80	1/5	30 s	1,564
8	gsm8k	1.00	0/5	23 s	1,564
9	tau2	0.90	0/10	803 s	988,110
10	tau2	0.75	0/20	436 s	1,703,460
11	appworld	0.00	2/5	2433 s	1,387,383
12	appworld	0.00	3/20	1818 s	5,211,584

gsm8k does not quite saturate here: 5 of 8 legs at 1.00

#4's gpt-4.1 swap and #7 lose one task of five, #6 two — five of 86. Read the CAUSE, not the rate: one of #6's was a dropped socket, not the model. The runner retries those now.

tau2 is the leg that moves

0.90 and 0.75 here, 1.00 and 0.80 on KinD. Episodes are nondeterministic: at n=10 one task is worth 0.10, so it cannot resolve less.

appworld 0.00 is the honest result

Runs complete, tokens record, evaluation says the goal was not met — 34 of 35 tasks self-declare finish. The per-task TIMEOUT dominates: 4 of 25 tasks left no report row, so err reads run.json.

0 rows lost attribution, 0 tasks lost to the health probe

The dev145 agent's per-task GET /v1/models probe (10 s cap, no retry) killed 12 of 141 KinD tasks; dev146 fixed it. An earlier matrix's 15 damaged rows passed a tokens == 0 check as clean — hence the structural detector.

9.3M input / 683k output tokens

over 5,719 s of leg wall time — appworld alone is 71% of the input, on 21 of 137 rows.

9.1 OpenShift vs KinD — Like-for-Like

Same 12 request bodies, same Service version, verified-identical instance config

#	bench	pass OCP	pass KinD	in OCP	in KinD	input
1	gsm8k	1.00	1.00	320	320	identical
2	gsm8k	1.00	1.00	3,166	3,166	identical
3	gsm8k	1.00	1.00	15,684	15,684	identical
4	gsm8k	0.80	1.00	3,874	4,183	
5	gsm8k	1.00	1.00	1,564	1,564	identical
6	gsm8k	0.60	0.80	1,258	1,564	
7	gsm8k	0.80	1.00	1,564	1,564	identical
8	gsm8k	1.00	1.00	1,564	1,564	identical
9	tau2	0.90	1.00	988,110	1,019,232	
10	tau2	0.75	0.80	1,703,460	1,743,312	
11	appworld	0.00	0.00	1,387,383	840,030	
12	appworld	0.00	0.00	5,211,584	2,009,348	

7 of 12 pass rates identical, and all 5 deltas favour KinD

So read the per-CAUSE table instead of the rate: wrong answers split 2 (OCP) to 1 (KinD), per-task timeouts 4 to 11 — all appworld — and one dropped socket, OCP #6.

6 legs byte-identical on input tokens

320 / 3,166 / 15,684 / 1,564 x3. Deterministic task selection plus the same model means identical work — the strongest available proof this is a like-for-like comparison, not merely a similar one.

#7 is in that set at 0.80 vs 1.00

Identical input tokens with different pass rates is not a contradiction: same prompts, different answers. Tokens prove the WORK matched; they say nothing about whether it was right.

Wall time: 5,719 s OCP vs 5,975 s KinD

Within 5% overall, but individual legs differ by up to 2.3x in either direction, and appworld — which dominates the total — inverts: #11 is faster on KinD, #12 is slower.

0 lost-attribution rows and 0 probe failures on both sides

137 OCP rows against 130 KinD. The 7-row gap is appworld tasks that timed out before the harness wrote a row (4 OCP, 11 KinD) — not lost telemetry.

9.2 What AuthBridge Costs — and Why It Has No Single Number

The designed 13-leg experiment (n=50, crossover replicates, gateway cache spaced), v1.28

non-LLM s / task (steady)	OpenShift	KinD	OCP step	KinD step
baseline (AuthBridge off)	0.423	0.099	—	—
auth-only	14.101	0.152	+13.68	+0.05
ibac-only	14.148	1.688	+0.05	+1.54
full (auth + ibac)	14.446	1.824	+0.30	+0.14
full + ibac:observe	14.598	1.826	+0.15	+0.00

does the finding travel?	OpenShift	KinD	travels?
Which layer costs anything	the sidecar	the judge	NO
Serial tool calls judged (p=1)	10 / 10	10 / 10	YES
Judged / tool-call ratio, n=50	0.90–0.98	0.90–0.98	YES
tau2 measured / projected	0.31x	2.49x	NO
Between-deploy noise floor	0.22 s	0.155 s	YES
Sidecar CPU / memory	no data	no data	—

The two clusters disagree about WHICH layer costs anything

On OpenShift the whole expense is the sidecar's mere presence and the judge is noise; on KinD the sidecar is nearly free and the judge is all of it. Both are right about their own cluster, and the same condition on the same image measures 4–93x apart. Never quote an absolute per-task plugin figure without naming the cluster.

Not a timeout — we checked

A timeout piles values on a round number. OpenShift's auth-only runs 8.05–17.93 s, SD 1.92, broad and unimodal: contention and queueing. On the quiet cluster the tail moves to full (median 1.82 s, max 14.00 s) because the judge is itself an LLM call — so the sidecar costs reproducibility, not only latency.

One flag decides whether the study measures anything

Every gsm8k leg sends the SAME 50 prompts and the gateway replays completions for ~10 min: unspaced, later legs measure the cache, in run order — the shape of a plugin effect. BM_CACHE_GAP=900 spaces them. The check needs no statistics: the conditions NEST, so no sidecar leg can beat a sidecar-free one. Clear by 41.8x (OCP), 1.4x (KinD).

The deploy is the unit of replication, not the task

A per-task interval measures variance WITHIN one deploy. The honest floor is between two deploys of one condition, and it exceeds every step but the dominant layer — so presets cannot be RANKED at any task count; add deploys, not tasks. Cut the other way: at $\sigma \approx 0.23$ s two deploys already resolve a 1 s effect, so the small steps are genuinely small rather than unresolved.

10. What the Service Can & Cannot Enact

The HTTP-only boundary, made explicit

✓ Enacts over HTTP

- Deploy MCP tool + A2A agent (CPU/mem, image, env)
- Deploy-time model swap (per-experiment agent)
- authbridge_enabled → inject the sidecar (layer-2)
- plugin_preset / plugins / on_error → AuthBridge layer-3
→ pluginPreset/plugins/onError; operator renders ConfigMap
- Run benchmark sessions; collect pass/fail + latency
- Emit + read MLflow traces; export to S3
- Service-owned config: MLflow + S3 via PUT /config
benchmarker only

✗ Out-of-band (reports, doesn't do)

- Cluster Secrets (hf-secret, openai-secret)
operator provisions; run precheck returns 424 naming it
- AuthBridge CLUSTER config — ibac judgeEndpoint / judgeModel
in the platform-config ConfigMap, not the agent API
- Any cluster-level API call — Rossoctl does these server-side

Principle:

- Never silently ignore an un-enactable request —
precheck (424) or reject (422), with an actionable reason

Cluster-agnostic by construction: works on kind / vanilla k8s / OpenShift; cross-cluster runs use per-instance route templates + a reachable internal issuer.